GPU – WARP LEVEL PRIMITIVES (FEAT. TENSOR CORES)

2026 SPRING

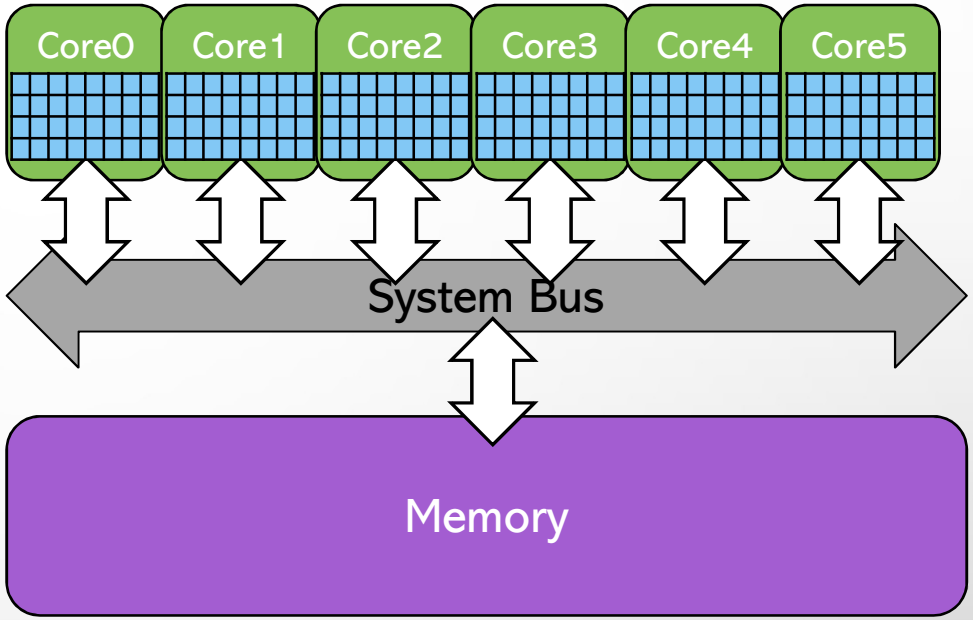
JINHO LEE

ECE, SNU

Accelerated Intelligent Systems Lab.

SIMT VS 32-WAY SIMD

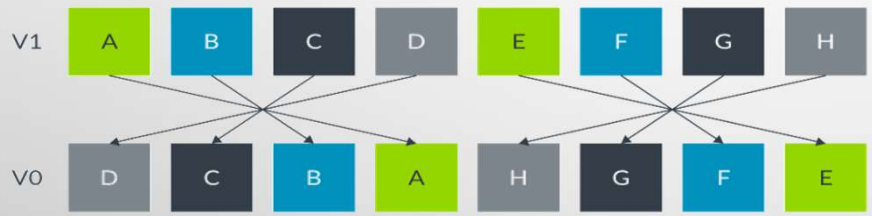
- We know that a warp is similar to a 32-way SIMD
- CUDA SIMT model hides it by calling each way a 'thread'
- But SIMT seems to only allow element-wise operations



```
int i = threadIdx.x + blockDim.x * blockIdx.x;
    if(i<n) C_d[i] = A_d[i] + B_d[i];
}
```

FLASHBACK ON SIMD INTRINSICS

- SIMD intrinsic supports much more than just element-wise operations
- Horizontal reduction
  - `C[i][j] = vaddvq_u32(vecC)`
- MAC with broadcast
  - `vmlaq_laneq_s32 (a, b, v, lane)`
- Permutation



- etc

WARP LEVEL PRIMITIVES

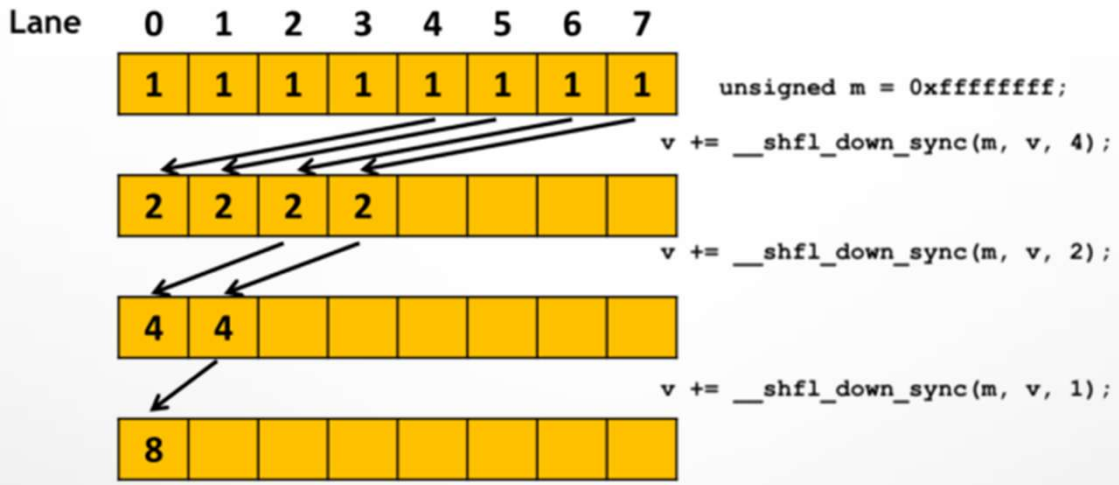
- Warp shuffle functions
- Warp vote functions
- Warp match functions
- Warp reduce functions
- Warp matrix functions

WARP PRIMITIVE STRUCTURE

```
T1 __warp_primitive(unsigned mask, T2 value, …);
```

- mask : represents the threads to be co-operated for __warp_primitive
  - Bitmap for which thread is participating
  - *"A bit, representing the thread's lane id, must be set for each participating thread to ensure they are properly converged before the intrinsic is executed by the hardware. <u>Each calling thread must have its own bit set in the mask and all non-exited threads named in mask must execute the same intrinsic with the same mask, or the result is undefined</u>."*
- value : the target value for __warp_primitive
- … : additional input arguments

WARP SHUFFLE



- Data exchange is performed <u>between registers</u>.
- better VS shared memory use: load + store + extra register to hold memory addr.

```
T __shfl_sync(unsigned mask, T var, int srcLane, int width=warpSize);

T __shfl_up_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);

T __shfl_down_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);

T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width=warpSize);
```

BROADCAST ACROSS WARP

```
__global__ void bcast(int arg) {
    int laneId = threadIdx.x & 0x1f;
    int value;
    if (laneId == 0)        // Note unused variable for
        value = arg;        // all threads except lane 0
    value = __shfl_sync(0xffffffff, value, 0);
   // Synchronize all threads in warp, and get "value" from lane 0
    if (value != arg)
        printf("Thread %d failed.\n", threadIdx.x);
}
```

Lower 5 bits for 32 threads

```
T __shfl_sync(unsigned mask, T var, int srcLane, int width=warpSize);
```

BROADCAST ACROSS WARP

```
__global__ void bcast(int arg) {
    int laneId = threadIdx.x & 0x1f;
    int value;
    if (laneId == 0)        // Note unused variable for
        value = arg;        // all threads except lane 0
    value = __shfl_sync(0xffffffff, value, 0);
   // Synchronize all threads in warp, and get "value" from lane 0
    if (value != arg)
        printf("Thread %d failed.\n", threadIdx.x);
}
```

Mask for 32 threads
(0xffffffff will be the most common – everyone)

BROADCAST ACROSS WARP

```
__global__ void bcast(int arg) {
    int laneId = threadIdx.x & 0x1f;
    int value;
    if (laneId == 0)        // Note unused variable for
        value = arg;        // all threads except lane 0
    value = __shfl_sync(0xffffffff, value, 0);
   // Synchronize all threads in warp, and get "value" from lane 0
    if (value != arg)
        printf("Thread %d failed.\n", threadIdx.x);
}
```

receiving value from lane 0

BROADCAST ACROSS WARP

```
__global__ void bcast(int arg) {
    int laneId = threadIdx.x & 0x1f;
    int value;
    if (laneId == 0)        // Note unused variable for
        value = arg;        // all threads except lane 0
    value = __shfl_sync(0xffffffff, value, 0);
   // Synchronize all threads in warp, and get "value" from lane 0
    if (value != arg)
        printf("Thread %d failed.\n", threadIdx.x);
}
```

__syncwarp() is not needed

BROADCAST ACROSS WARP

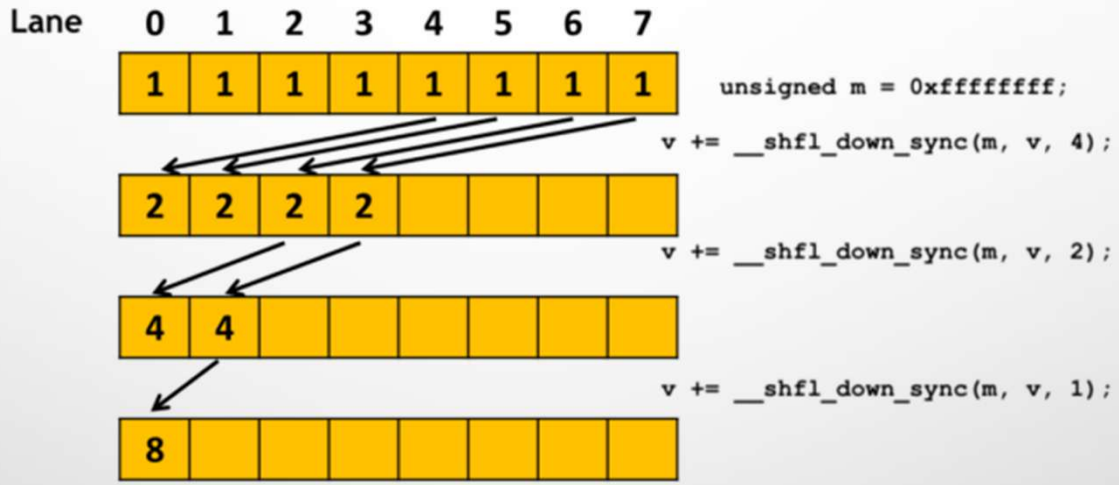
```
__global__ void bcast(int arg) {
    int laneId = threadIdx.x & 0x1f;
    int value;
    if (laneId == 0)        // Note unused variable for
        value = arg;        // all threads except lane 0
    value = __shfl_sync(0xffffffff, value, 0);
   // Synchronize all threads in warp, and get "value" from lane 0
    if (value != arg)
        printf("Thread %d failed.\n", threadIdx.x);
}
```

__syncwarp() is not needed

__SHFL_DOWN_SYNC

- Each lane will receive from `delta` lanes above

```
T __shfl_down_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);
```



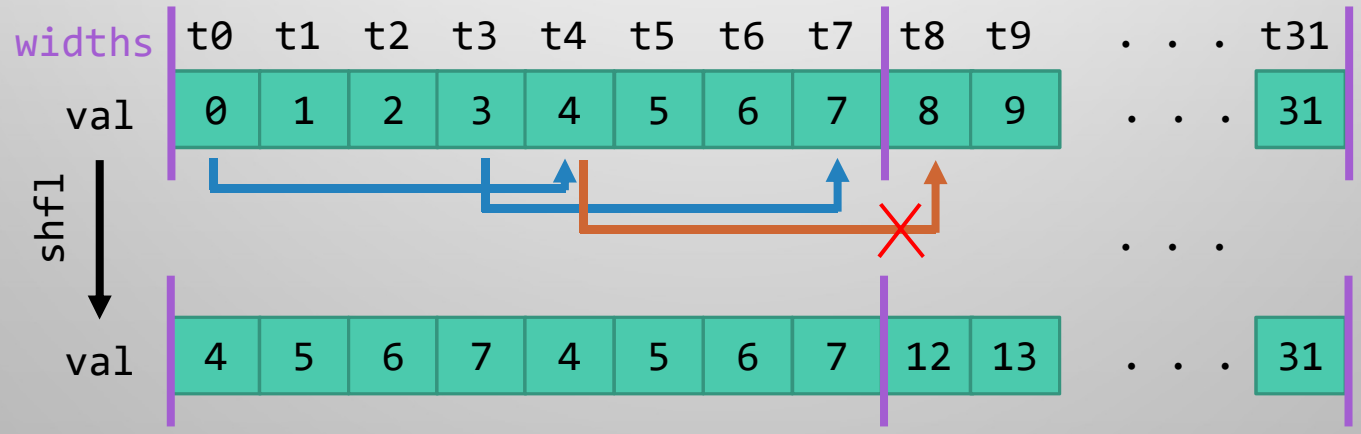

```
template <typename T>
__device__ T reduce_add_shfl(T val) {
    for (int offset = 16; offset > 0; offset /= 2) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }
    return val;
}
```

SHUFFLE WIDTH

- `width`: power of 2 in [1, 32]. Used to divide threads to separate entities (subwarps) of size `width`.

```
T __shfl_sync(unsigned mask, T var, int srcLane, int width=warpSize);
```

```
__global__ void shfl_width() {
    int val = threadIdx.x;
    val = __shfl_down_sync(0xffffffff, val, 4/*delta*/, 8/*widths*/);
    printf("t%d: %d\n", threadIdx.x, val);
}
```



WARP SHUFFLE

```
T __shfl_up_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);

// receive from lower-numbered lanes


T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width=warpSize);

// receive from “laneid xor laneMask” lane
```

WARP REDUCE

```
unsigned __reduce_add_sync(unsigned mask, unsigned value);
unsigned __reduce_min_sync(unsigned mask, unsigned value);
unsigned __reduce_max_sync(unsigned mask, unsigned value);
int __reduce_add_sync(unsigned mask, int value);
int __reduce_min_sync(unsigned mask, int value);
int __reduce_max_sync(unsigned mask, int value);

unsigned __reduce_and_sync(unsigned mask, unsigned value);
unsigned __reduce_or_sync(unsigned mask, unsigned value);
unsigned __reduce_xor_sync(unsigned mask, unsigned value);
```

- Reduction within warp is separately provided
- w/ various reduce operators
- Currently, the only supported types are unsigned and int

SETTING THE MASK

```
template <typename T>
__device__ T reduce_add_shfl(T val) {
    for (int offset = 16; offset > 0; offset /= 2) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }
    return val;
}
```

- Setting mask
  - Option 1: Manual setting
  - Option 2: Using Warp Vote functions
  - Option 3: Using warp match functions

SETTING THE MASK

- Setting `mask`
  - Option 1: Manual setting
  - Option 2: Using Warp Vote functions
  - Option 3: Using warp match functions

```
__global__ void mask_manual_setting() {
    const unsigned mask = 3; // equivalent to 0b00....011
    int val = 1;
    if (threadIdx.x < 2) {
        val = __reduce_add_sync(mask, val);
    }
    if (threadIdx.x == 0) printf("Total sum: %d\n", val);
    // Total sum: 2
}
```

SETTING THE MASK

- Setting mask
  - Option 1: Manual setting – error prone & tedious manual mask setting
  - Option 2: Using Warp Vote functions
  - Option 3: Using warp match functions

```
__global__ void mask_manual_setting() {
    const unsigned mask = 3; // equivalent to 0b00....011
    int val = 1;
    if (threadIdx.x > 2) {
        val = __reduce_add_sync(mask, val);
    }
    if (threadIdx.x == 0) printf("Total sum: %d\n", val);
    // Total sum: 2
}
```

```
Kernel launch failed: an illegal
instruction was encountered
```

SETTING THE MASK

- Setting mask
  - Option 1: Manual setting
  - Option 2: Using Warp Vote functions
  - Option 3: Using warp match functions

```
__global__ void mask_vote() {
    int val = 1;
    if (threadIdx.x < 2) {
        unsigned mask = __activemask();
        val = __reduce_add_sync(mask, val);
    }
    if (threadIdx.x == 0) printf("Total sum: %d\n", val);
    // Total sum: 2
}
```

SETTING THE MASK

- Setting mask
  - Option 1: Manual setting
  - Option 2: Using Warp Vote functions
  - Option 3: Using warp match functions

```
__global__ void mask_match() {
    int val = 1;
    unsigned mask = __match_any_sync(0xffffffff, threadIdx.x < 2);
    val = __reduce_add_sync(mask, val);
    if (threadIdx.x == 0) printf("Total sum: %d\n", val);
    //Total sum: 2
}
```

WARP VOTE FUNCTIONS

```
int __all_sync(unsigned mask, int predicate);
int __any_sync(unsigned mask, int predicate);
unsigned __ballot_sync(unsigned mask, int predicate);
unsigned __activemask();
```

- Allows threads to perform a reduction-and-broadcast operation.
  - `__ballot_sync`: returns an integer whose Nth bit is set iff predicate evaluates to non-zero for the Nth thread of the warp and the Nth thread is active.
  - `__activemask`: returns a 32-bit integer mask of all currently active threads in the calling warp.

WARP MATCH FUNCTIONS

```
unsigned int __match_any_sync(unsigned mask, T value);
unsigned int __match_all_sync(unsigned mask, T value, int *pred);
```

- Allows threads to perform a broadcast-and-compare.
  - `__match_any_sync`: returns mask of threads that have same value of `value` in `mask`.
  - `__match_all_sync`: Returns `mask` if all threads in `mask` have the same value for value; otherwise 0 is returned. Predicate `pred` is set to `true` if all threads in mask have the same value of value; otherwise set to `false`.

WARP MATCH FUNCTIONS

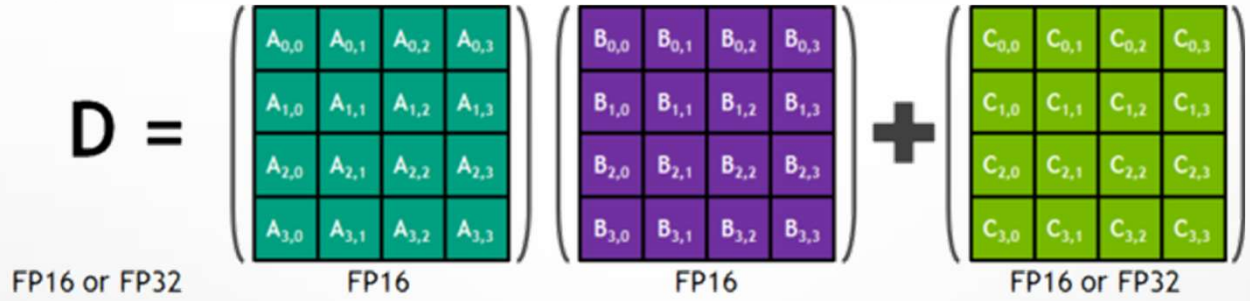
```
__global__ void mask_match() {
    int val = 1;
    unsigned mask = __match_any_sync(0xffffffff, threadIdx.x < 2);
    val = __reduce_add_sync(mask, val);
    if (threadIdx.x == 0 || threadIdx.x == 31) printf("Total sum from t%d: %d\n",
    threadIdx.x, val);
}
```

```
Total sum from t0: 2
Total sum from t31: 30
```

WARP MATRIX AND TENSORCORE

- Tensorcore: dedicated hardware that lets matmul
- Executed through warp matrix primitives

WARP MATRIX FUNCTION AND TENSORCORE



- Accelerates matrix problems of the form of `D=A*B+C` with Tensor Cores.
- The only similarity with prior warp-level primitives is that the functions are run <u>within a single warp</u>.
- Important: requires <u>co-operation from all 32 threads in a warp</u>, otherwise the code execution is likely to hang.

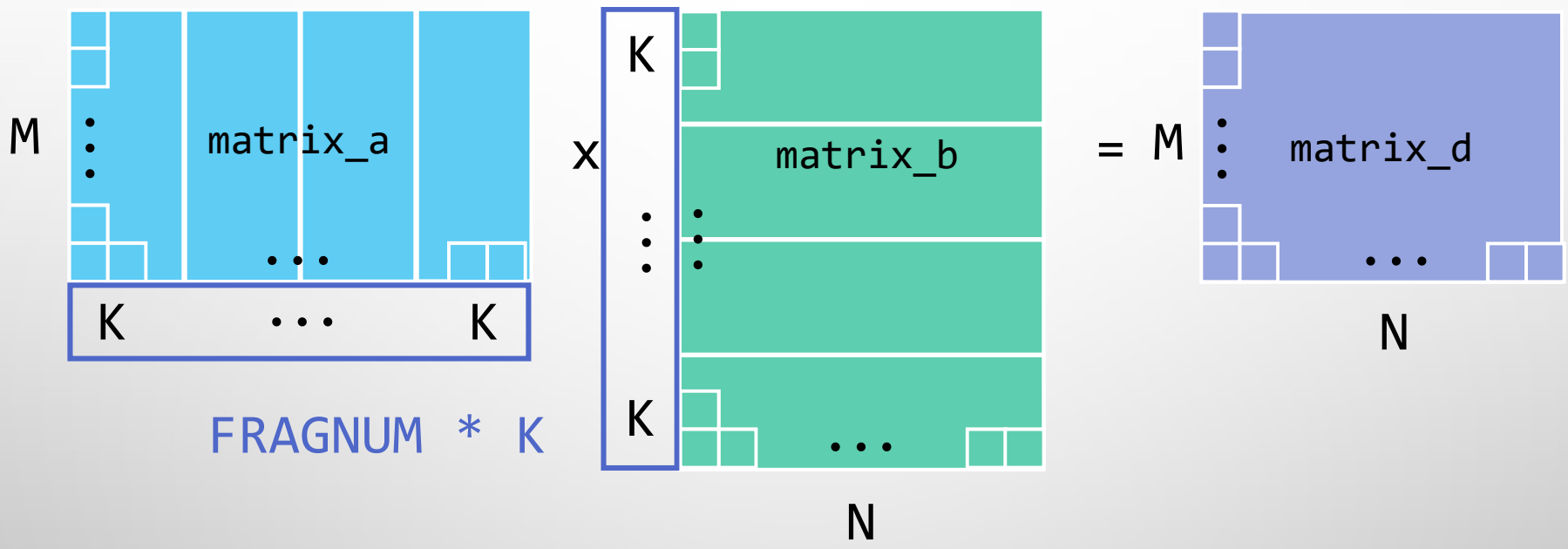
WARP MATRIX FUNCTION AND TENSORCORE

- Headers and namespaces
  - The complete namespace is `nvcuda::wmma::*`, but for clarity we keep `wmma` in the code.

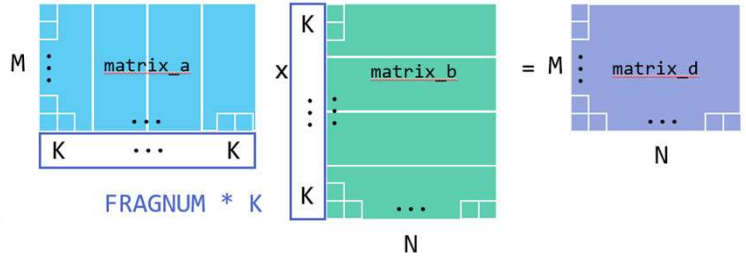
```
#include <mma.h>
using namespace nvcuda;
```

- Declarations and initialization
  - Use **`fragments`** to store inputs and outputs of MMA (Matrix Multiply Accumulate).
  - `fragment`: overloaded template class containing a section of a matrix distributed across all threads in a warp.

MATRIX DIVIDED INTO FRAGMENTS

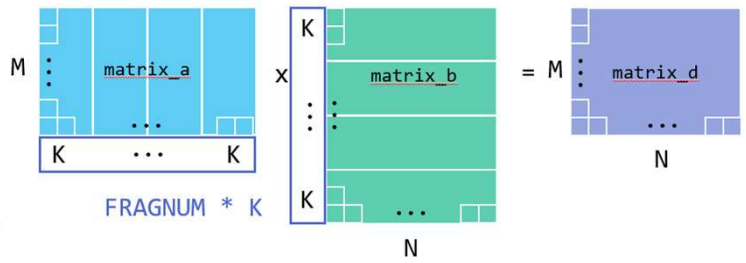


WARP MATRIX FUNCTION




```
#define M 8
#define N 32
#define K 16
#define FRAGNUM 4

__global__ void matmul_kernel_char(unsigned char* matrix_a, unsigned char* matrix_b, int* matrix_d) {
   if (threadIdx.x < 32) {
        wmma::fragment<wmma::matrix_a, M, N, K, unsigned char, wmma::row_major> frag_a;
        wmma::fragment<wmma::matrix_b, M, N, K, unsigned char, wmma::row_major> frag_b;
        wmma::fragment<wmma::accumulator, M, N, K, int> frag_d;

        wmma::fill_fragment(frag_d, 0);

        for (int i = 0; i < FRAGNUM; i++) {
            wmma::load_matrix_sync(frag_a, matrix_a + K * i, FRAGNUM * K);
            wmma::load_matrix_sync(frag_b, matrix_b + N * K * i, N);

            wmma::mma_sync(frag_d, frag_a, frag_b, frag_d);
        }

        wmma::store_matrix_sync(matrix_d, frag_d, N, wmma::mem_row_major);
   }
}
```

WARP MATRIX FUNCTION



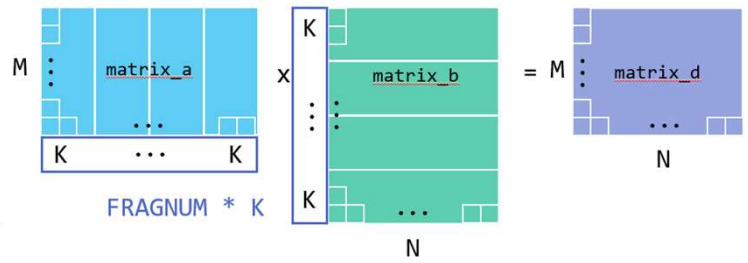

```
#define M 8
#define N 32
#define K 16
#define FRAGNUM 4

__global__ void matmul_kernel_char(unsigned char* matrix_a, unsigned char* matrix_b, int* matrix_d) {
   if (threadIdx.x < 32) {
        wmma::fragment<wmma::matrix_a, M, N, K, unsigned char, wmma::row_major> frag_a;
        wmma::fragment<wmma::matrix_b, M, N, K, unsigned char, wmma::row_major> frag_b;
        wmma::fragment<wmma::accumulator, M, N, K, int> frag_d;
```

<Defining fragments>

```
template<typename Use, int m, int n, int k, typename T,
typename Layout=void> class fragment;
```

- Use
  - matrix_a : for first multiplicand A
  - matrix_b : for second multiplicand B
  - accumulator : for source/destination accumulators C/D

WARP MATRIX FUNCTION




```
#define M 8
#define N 32
#define K 16
#define FRAGNUM 4

__global__ void matmul_kernel_char(unsigned char* matrix_a, unsigned char* matrix_b, int* matrix_d) {
   if (threadIdx.x < 32) {
        wmma::fragment<wmma::matrix_a, M, N, K, unsigned char, wmma::row_major> frag_a;
        wmma::fragment<wmma::matrix_b, M, N, K, unsigned char, wmma::row_major> frag_b;
        wmma::fragment<wmma::accumulator, M, N, K, int> frag_d;
```

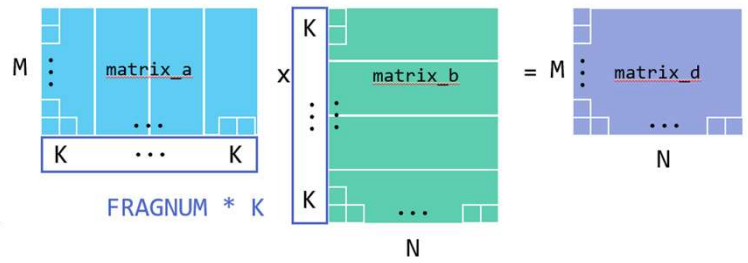
<Defining fragments>

`template<typename Use, int m, int n, int k, typename T, typename Layout=void> class fragment;`

- `Int m, n, k*`
  - `matrix_a` : size (m, k)
  - `matrix_b` : size (k, n)
  - `accumulator` : size (m, n)

* represents the sizes of a `fragment`, not the entire matrix.

WARP MATRIX FUNCTION




```
#define M 8
#define N 32
#define K 16
#define FRAGNUM 4

__global__ void matmul_kernel_char(unsigned char* matrix_a, unsigned char* matrix_b, int* matrix_d) {
   if (threadIdx.x < 32) {
        wmma::fragment<wmma::matrix_a, M, N, K, unsigned char, wmma::row_major> frag_a;
        wmma::fragment<wmma::matrix_b, M, N, K, unsigned char, wmma::row_major> frag_b;
        wmma::fragment<wmma::accumulator, M, N, K, int> frag_d;
```

<Defining fragments>

`template<typename Use, int m, int n, int k, typename T, typename Layout=void> class fragment;`

- `typename T`
  - Only allows a fixed set of types and `fragment` sizes.

WARP MATRIX FUNCTION

- Element types and matrix sizes
  - Only a fixed configurations are possible, otherwise causes compile error.

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
__half	__half	float	16x16x16
__half	__half	float	32x8x16
__half	__half	float	8x32x16
__half	__half	__half	16x16x16
__half	__half	__half	32x8x16
__half	__half	__half	8x32x16
unsigned char	unsigned char	int	16x16x16
unsigned char	unsigned char	int	32x8x16
unsigned char	unsigned char	int	8x32x16
signed char	signed char	int	16x16x16
signed char	signed char	int	32x8x16
signed char	signed char	int	8x32x16

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#element-types-and-matrix-sizes

WARP MATRIX FUNCTION

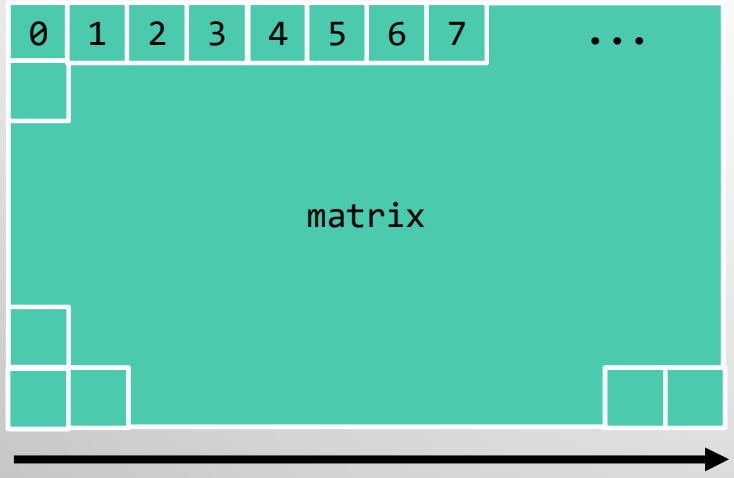
- Element types and matrix sizes
  - Alternative floating points are also supported (ex. bf16, tf32).
  - For more details, refer to the official doc.

Matrix A	Matrix B	Accumulator	Matrix Size (m-n-k)
__nv_bfloat16	__nv_bfloat16	float	16x16x16
__nv_bfloat16	__nv_bfloat16	float	32x8x16
__nv_bfloat16	__nv_bfloat16	float	8x32x16
precision::tf32	precision::tf32	float	16x16x8

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#alternate-floating-point

WARP MATRIX FUNCTION

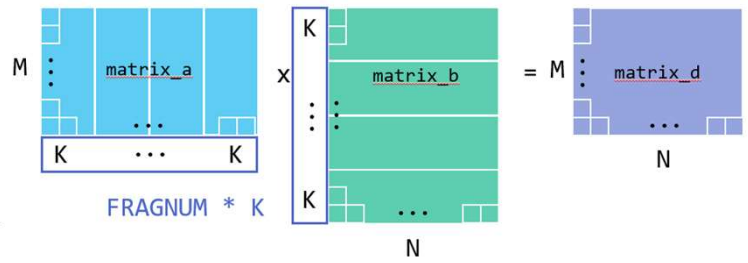
- Row major
  - Index: the order the elements are stored in memory



- Column major

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#warp-matrix-functions

WARP MATRIX FUNCTION



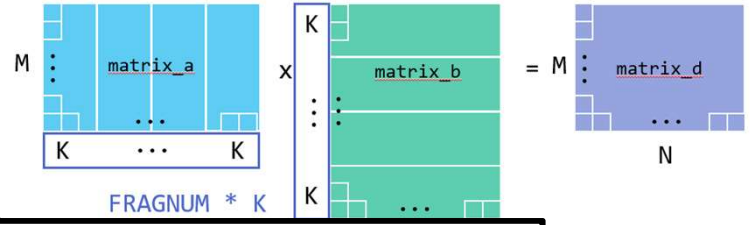

```
#define M 8
#define N 32
#define K 16
#define FRAGNUM 4

__global__ void matmul_kernel_char(unsigned char* matrix_a, unsigned char* matrix_b, int* matrix_d) {
   if (threadIdx.x < 32) {
        wmma::fragment<wmma::matrix_a, M, N, K, unsigned char, wmma::row_major> frag_a;
        wmma::fragment<wmma::matrix_b, M, N, K, unsigned char, wmma::row_major> frag_b;
        wmma::fragment<wmma::accumulator, M, N, K, int> frag_d;

        wmma::fill_fragment(frag_d, 0);
```

<Filling fragment with a constant>

`void fill_fragment(fragment<...> &a, const T& v);`

- Fills the fragment with a constant value v.

WARP MATRIX FUNCTION



<Loading matrix values from memory to fragment>

```
void load_matrix_sync(fragment<...> &a, const T* mptr, unsigned ldm, layout_t layout);
```

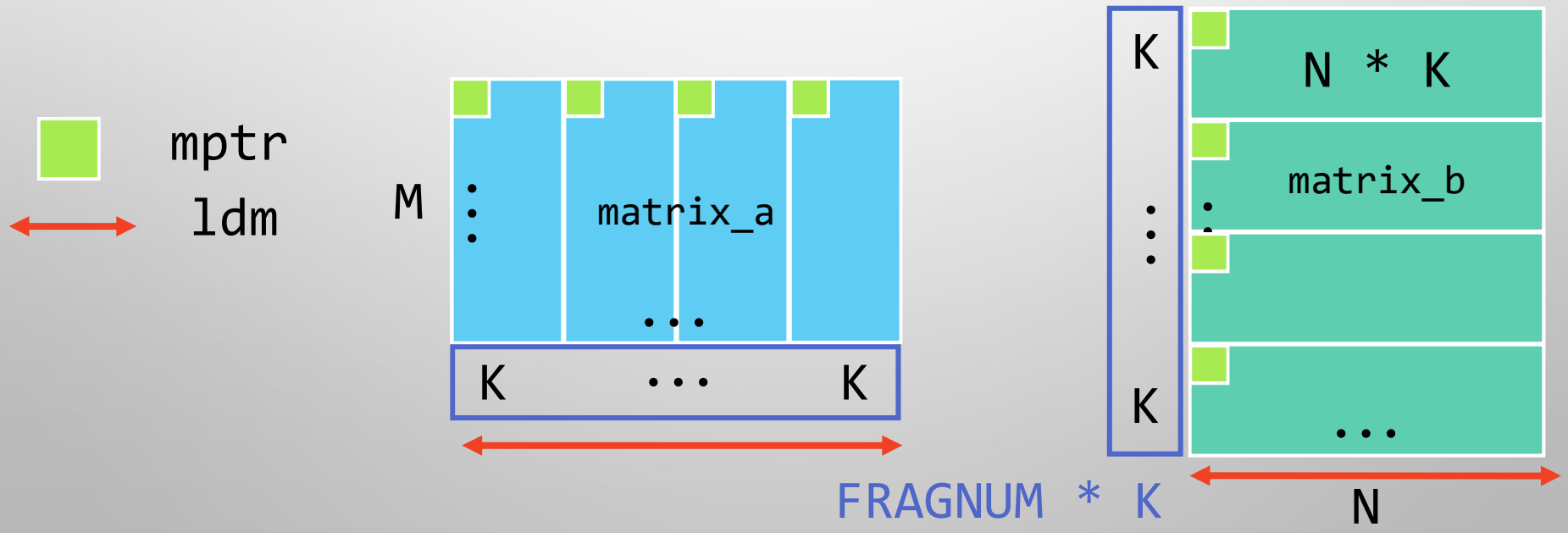
- `Mptr` : pointer pointing to the first element of the matrix in memory
- `ldm` : stride in elements between consecutive rows/columns (for row/column major layout)
- `layout` : should be defined for `accumulator`, either `mem_row_major/mem_col_major`

```
        for (int i = 0; i < FRAGNUM; i++) {
            wmma::load_matrix_sync(frag_a, matrix_a + K * i, FRAGNUM * K);
            wmma::load_matrix_sync(frag_b, matrix_b + N * K * i, N);

            wmma::mma_sync(frag_d, frag_a, frag_b, frag_d);
        }

        wmma::store_matrix_sync(matrix_d, frag_d, N, wmma::mem_row_major);
    }
}
```

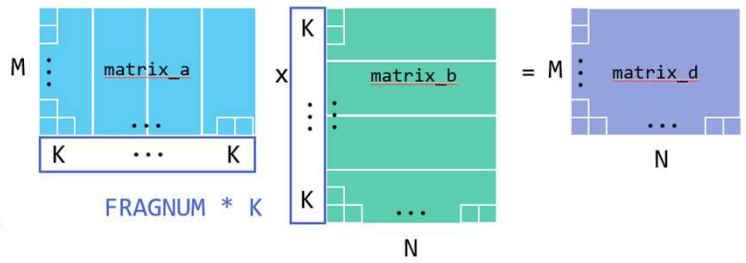
WARP MATRIX FUNCTION

- Visualization of `mptr` and `ldm`

```
for (int i = 0; i < FRAGNUM; i++) {
    wmma::load_matrix_sync(frag_a, matrix_a + K * i, FRAGNUM * K);
    wmma::load_matrix_sync(frag_b, matrix_b + N * K * i, N);

    wmma::mma_sync(frag_d, frag_a, frag_b, frag_d);
}
```



WARP MATRIX FUNCTION



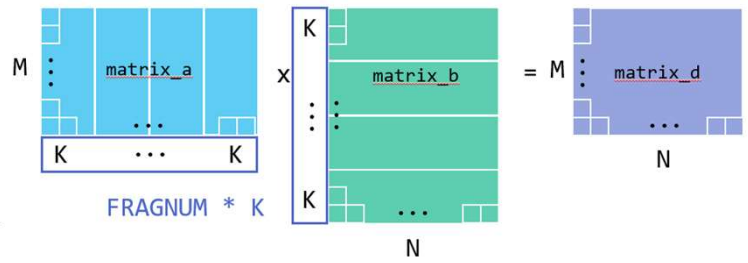
<Matrix multiplication with Tensor Cores>

```
void mma_sync(fragment<...> &d, const fragment<...> &a, const
fragment<...> &b, const fragment<...> &c, bool satf=false);
```

- performs the warp-synchronous matrix multiply-accumulate operation D=A*B+C (In this example, in-place operation of D=A*B+D is performed).

```
        wmma::fill_fragment(frag_d, 0);

        for (int i = 0; i < FRAGNUM; i++) {
            wmma::load_matrix_sync(frag_a, matrix_a + K * i, FRAGNUM * K);
            wmma::load_matrix_sync(frag_b, matrix_b + N * K * i, N);

            wmma::mma_sync(frag_d, frag_a, frag_b, frag_d);
        }

        wmma::store_matrix_sync(matrix_d, frag_d, N, wmma::mem_row_major);
    }
}
```

WARP MATRIX FUNCTION




```
#define M 8
#define N 32
```

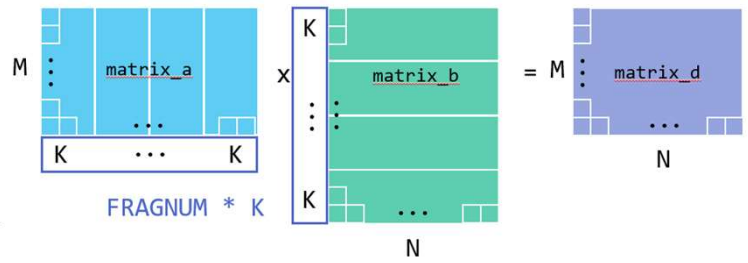
<Storing results to memory>

```
void store_matrix_sync(T* mptr, const fragment<...> &a, unsigned ldm,
layout_t layout);
```

- stores the matrix fragment a to memory.

```
        wmma::fill_fragment(frag_d, 0);

        for (int i = 0; i < FRAGNUM; i++) {
            wmma::load_matrix_sync(frag_a, matrix_a + K * i, FRAGNUM * K);
            wmma::load_matrix_sync(frag_b, matrix_b + N * K * i, N);

            wmma::mma_sync(frag_d, frag_a, frag_b, frag_d);
        }

        wmma::store_matrix_sync(matrix_d, frag_d, N, wmma::mem_row_major);
    }
}
```

WARP MATRIX FUNCTION




```
#define M 8
#define N 32
#define K 16
#define FRAGNUM 4

__global__ void matmul_kernel_char(unsigned char* matrix_a, unsigned char* matrix_b, int* matrix_d) {
  if (threadIdx.x < 32) {
        wmma::fragment<wmma::matrix_a, M, N, K, unsigned char, wmma::row_major> frag_a;
        wmma::fragment<wmma::matrix_b, M, N, K, unsigned char, wmma::row_major> frag_b;
        wmma::fragment<wmma::accumulator, M, N, K, int> frag_d;

        wmma::fill_fragment(frag_d, 0);

        for (int i = 0; i < FRAGNUM; i++) {
            wmma::load_matrix_sync(frag_a, matrix_a + K * i, FRAGNUM * K);
            wmma::load_matrix_sync(frag_b, matrix_b + N * K * i, N);

            wmma::mma_sync(frag_d, frag_a, frag_b, frag_d);
        }

        wmma::store_matrix_sync(matrix_d, frag_d, N, wmma::mem_row_major);
   }
}
```

a simple 1-warp version.
more index calculation will be needed for utilizing multiple warps

SUMMARY

- warp-level primitive provides more tools for 32-way SIMD like operations
- Shuffle and reduce are useful, and many mask-generating primitives exist
- WMMA can be used to utilize the tensorcores